Lessons from Log4Shell: building a CRA-ready Log4j

Piotr Karwasz, VP Logging, Apache Software Foundation: pkarwasz@apache.org
Piotr Karwasz, freelancer: piotr@copernik.eu

Who are we?

The Apache Software Foundation:

- 501(c)**3** foundation
- Only individual, no organizational members
- 100% volunteer-based
- 210 **independent** PMCs
- Foundation dictates governance rules

Apache Logging Services PMC:

- Log4cxx, Log4j and Log4net: logging libraries for various languages
- A dozen active members

Piotr Karwasz:

- PhD in Mathematics (2009)
- École Normale Supérieure (Paris)
- Father of three daughters:
  Mimi, Lili and Nati.
- 2017: IT freelancer
- 2022, January: start contributing to Log4j
- 2022, July: Logging Services PMC member

Remember,
remember,
the 9th of December!
(2021)

Source: Devianart

What happened?

- Log4Shell (CVE-2021-44228), a vulnerability in Log4j, became a global supply-chain incident
- It exposed a key problem: many organizations did not know whether they were using Log4j
- It pushed users to migrate from Log4j 1 (EOL since 2015) to Log4j 2
- It helped accelerate new software security regulations, such as the Cyber Resilience Act

Lessons learned in Log4j

Supply chain problems:

- Complex release process,
- Slow builds and flaky tests,
- Dependency updates not automated

Too many bundled features:

learn to say NO (intelligently).

Documentation problems:

- Hard to read
- Missing best practices
- Many undocumented features

Improvement directions

Main improvement directions:

1. Documentation
2. Release process
3. Machine-readable metadata
4. Vulnerability handling
5. Community sustainability

These changes try to cover (and often exceed):

- The “due diligence” requirements of our users,
- The proposed CRA attestations

Documentation

Security through education

Logging is not always safe:

- Unstructured logging:
  CWE-93 CRLF Injection
- Presence of sensitive information in logs:
  CWE-215: SI in Debug Code
- Injection of {} Log4j formatting patterns:

```
String user = “root {}”;
String what = “login”;
log.(user + “failed to {}”, what);
```

- Reliable and secure transport.

Solutions:

- **Total rewrite** of documentation website.
  Learn from the source, not LLM.
- **Generation** of reference from code:
  Living documentation,
  Developers can not forget.
- Provide **best practices** and tips:
  The maintainers knowledge base was mainly unwritten.

Tip: there will be an in-depth book by Christian Grobmeier published by Manning.

http://logging.apache.org
Logging Services™

LOG4J 2™

Apache Log4j 2 ™ | Logging Wiki | Apache | Logging Services | Sonar | GitHub

APACHE LOG4J™ 2
- About
- Download
- Javadoc
- Maven, Ivy, Gradle Artifacts
- Runtime Dependencies
- Changelog
- FAQ
- Performance
- Articles and Tutorials
- Thanks

FOR CONTRIBUTORS
- Building Log4j from Source
- Guidelines
- Style Guide

MANUAL
- Introduction
- Architecture
- Log4j 1.x Migration
- Java API
- Scala API
- Configuration
  - Automatic Configuration
  - Additivity
  - Automatic Reconfiguration
  - Chainsaw Support
  - Configuration Syntax
  - XML Syntax
  - JSON Syntax
  - YAML Syntax
  - Properties Syntax
  - Configuring Loggers
  - Configuring Appenders
  - Configuring Filters
  - Property Substitution
  - Lookup Variables
  - Scripts
  - XInclude
  - Composite Configurations
  - Status Messages
  - Unit Testing in Maven
  - System Properties

Configuration

Inserting log requests into the application code requires a fair amount of planning and effort. Observation shows that approximately 4 percent of code is dedicated to logging. Consequently, even moderately sized applications will have thousands of logging statements embedded within their code. Given their number, it becomes imperative to manage these log statements without the need to modify them manually.

Configuration of Log4j 2 can be accomplished in 1 of 4 ways:

1. Through a configuration file written in XML, JSON, YAML, or properties format.
2. Programmatically, by creating a ConfigurationFactory and Configuration implementation.
3. Programmatically, by calling the APIs exposed in the Configuration interface to add components to the default configuration.
4. Programmatically, by calling methods on the internal Logger class.

This page focuses primarily on configuring Log4j through a configuration file. Information on programmatically configuring Log4j can be found at Extending Log4j 2 and Programmatic Log4j Configuration.

Note that unlike Log4j 1.x, the public Log4j 2 API does not expose methods to add, modify or remove appenders and filters or manipulate the configuration in any way.

Automatic Configuration

Log4j has the ability to automatically configure itself during initialization. When Log4j starts it will locate all the ConfigurationFactory plugins and arrange them in weighted order from highest to lowest. As delivered, Log4j contains four ConfigurationFactory implementations: one for JSON, one for YAML, one for properties, and one for XML.

1. Log4j will inspect the `"log4j.configurationFile"` system property and, if set, will attempt to load the configuration using the `ConfigurationFactory` that matches the file extension.
2. If no system property is set the properties ConfigurationFactory will look for `log4j2-test.properties` in the classpath.
3. If no such file is found the YAML ConfigurationFactory will look for `log4j2-test.yaml` or `log4j2-test.yml` in the classpath.
4. If no such file is found the JSON ConfigurationFactory will look for `log4j2-test.json` or `log4j2-test.jsn` in the classpath.
5. If no such file is found the XML ConfigurationFactory will look for `log4j2-test.xml` in the classpath.
6. If a test file cannot be located the properties ConfigurationFactory will look for `log4j2.properties` on the classpath.
7. If a properties file cannot be located the YAML ConfigurationFactory will look for `log4j2.yaml` or `log4j2.yml` on the classpath.
8. If a YAML file cannot be located the JSON ConfigurationFactory will look for `log4j2.json` or `log4j2.jsn` on the classpath.
9. If a JSON file cannot be located the XML ConfigurationFactory will try to locate `log4j2.xml` on the classpath.
10. If no configuration file could be located the `DefaultConfiguration` will be used. This will cause logging output to go to the console.

An example application named `MyApp` that uses log4j can be used to illustrate how this is done.

```
1.  import com.foo.Bar;
2.
3.  // Import log4j classes.
4.  import org.apache.logging.log4j.Logger;
5.  import org.apache.logging.log4j.LogManager;
6.
7.  public class MyApp {
8.
9.      // Define a static logger variable so that it references the
10.     // Logger instance named "MyApp".
11.     private static final Logger logger = LogManager.getLogger(MyApp.class);
12.
13.     public static void main(final String... args) {
14.
15.         // Set up a simple configuration that logs on the console.
16.
17.         logger.trace("Entering application.");
```

Old website

LOG4J 2 — a subproject of Apache Logging Services

Home
- Download
- Support
- Manual
  - Getting started
  - Installation
  - API
  - Implementation
    - Architecture
    - Configuration
      - **Configuration file**
      - Configuration properties
      - Programmatic configuration
      - Appenders
      - Layouts
      - Lookups
      - Filters
      - Scripts
      - JMX
    - Extending
    - Performance
- References
- Resources
- Components
- Related projects

ments:

Loggers

Loggers are the entry point of the logging pipeline, which is directly used in the code. Their configuration must specify which level of messages they log and to which appenders they send the messages. We will cover them while configuring loggers.

Appenders

Appenders are the exit point of the logging pipeline. They decide which resource (console, file, database, or similar) the log event is sent to. In the examples of this chapter, we will only use the console appender and the file appender.

Layouts

Layouts tell appenders how to format the log event: text, JSON, XML, or similar. In the examples of this chapter, we will only use Pattern Layout and JSON Template Layout.

A moderately complex configuration might look like this:

XML | JSON | YAML | Properties

Snippet from an example ***log4j2.xml***

```
<?xml version="1.0" encoding="UTF-8"?>
<Configuration xmlns="https://logging.apache.org/xml/ns"
               xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
               xsi:schemaLocation="
                   https://logging.apache.org/xml/ns
                   https://logging.apache.org/xml/ns/log4j-config-2.xsd">
  <Appenders>
    <Console name="CONSOLE">  ①
      <PatternLayout pattern="%p - %m%n"/>
    </Console>
    <File name="MAIN" fileName="logs/main.log">  ②
      <JsonTemplateLayout/>
    </File>
    <File name="DEBUG_LOG" fileName="logs/debug.log">  ③
      <PatternLayout pattern="%d [%t] %p %c - %m%n"/>
    </File>
  </Appenders>
  <Loggers>
    <Root level="INFO">  ④
      <AppenderRef ref="CONSOLE" level="WARN"/>
      <AppenderRef ref="MAIN"/>
    </Root>
    <Logger name="org.example" level="DEBUG">  ⑤
      <AppenderRef ref="DEBUG_LOG"/>
    </Logger>
```

Contents

- Configuration file location
  - Predefined ConfigurationFactory plugins
- Syntax
- Main configuration elements
- Additional configuration elements
- Global configuration attributes
  - monitorInterval
  - status
- Loggers
  - name
  - additivity
  - level
  - includeLocation
  - Appender references
  - Additional context properties
  - Filters
- Appender references
  - ref
  - level
  - Filters
- Property substitution
  - Runtime property substitution
- Monitor Resources
- Arbiters
- Composite configuration
- Format specific notes
  - XML format
    - Global configuration attributes
      - schema
      - strict
    - XInclude
  - Java properties format
- Extending
  - Plugin preliminaries
  - Extending ConfigurationFactory plugins
  - Plugins represented in a

New website

LOG4J

a subproject of Apache Logging Services

Home
- Download
- Support
  - Versioning and maintenance policy
  - Security
- Manual
- References
- Resources
- Components
- Related projects

Home / Support / Versioning and maintenance policy

Edit this Page

Versioning and maintenance policy

This page answers the following questions:

- How are Log4j releases versioned?
- How shall users align the Log4j API and Log4j Core versions?
- How does Log4j classify maintenance status of released versions?

Versioning policy

Since version `2.0`, Log4j follows semantic versioning, with release numbers of the form:

```
<major>.<minor>.<patch>[-<pre-release>]
```

where:

<major>

The major version number is incremented when **breaking changes** are introduced.

Upgrading to a new major version typically requires code changes in your application. For each major release, a migration guide is provided.

See Migrating from Log4j 1 for instructions on migrating from Log4j 1 to Log4j 2.

<minor>

The minor version number is incremented when new features are added in a backward-compatible manner, such as:

- New Java methods or classes added to the public API of one of the Log4j artifacts.
- New configuration attributes added to Log4j Plugins (appenders, layouts, filters, etc.).
- Functionality or Java methods/classes being deprecated.
- Behavioral changes introduced without breaking the public API.

Upgrading to a new minor version usually does not require code changes, unless you rely on undocumented behavior that has changed. To avoid accumulating such changes, we recommend upgrading

Contents

Versioning policy
Version alignment
Version lifecycle and maintenance policy

Versioning policy [BR.02]

Available versions

Below you can find the list of available Log4j versions and their associated maintenance status; Active Development, Active Maintenance, End-of-Maintenance, and End-of-Life. Refer to Versioning and maintenance policy for details.

Table 3. Maintenance status of selected Log4j versions

Version	Status	Latest release	First stable release	EOM	EOL	Notes
3.0.x	AD	`3.0.0-beta3`				
2.26.x	AD					
2.25.x	AM	`2.25.3`	2025-12-15			
2.24.x	EOM	`2.24.3`	2024-09-03	2025-06-13		
2.12.x	EOM	`2.12.4`	2019-06-23	2021-12-29		Last release supporting Java 7
2.3.x	EOM	`2.3.2`	2015-05-09	2021-12-29		Last release supporting Java 6
1.x	EOL	`1.2.17`	2000-01-08	2014-07-12	2015-08-05	Last release supporting Java 1.4

▸ Click to see all past versions

Support status [EL.01]

Results

- An increase in documentation PRs
- An increase in visits
- Better LLM results

Release process

Build process evolution

December 2021:

- Tests ran on Jenkins
- Built on maintainer machines
- Manual signing
- Non-reproducible builds
- Binary and site build time: **hours**

September 2023:

- Migrated to **GitHub Actions**
- Sequential tests
- 30-40% of failed test runs
- Binary build times: up to 60 minutes
- Site build times: less than **5 minutes**

September 2024:

- **Automated CI build** up to voting phase (first in ASF),
- **Reproducible** builds, **[BR.01]**
- Signed by **CI GPG key**, **[BR.03]**
- Parallel tests,
- **Dynamic tests (fuzzing)**,
- 8% of test runs fails (21% flaky),
- Build + deploy around 30 minutes.
- Searchable build failure database: Gradle Develocity

Planned for April 2026:

- Fully automated through: Apache Trusted Releases
- SLSA **build** attestations, **[BR.03]**
- SLSA **source** attestations (level 4: all commits with a non-author review). **[BR.03]**

← Back to Select a release

① COMPOSE → ② → ③

Compose Log4j 2.25.4

Manage the **candidate draft** for Log4j 2.25.4. Add files, review checks, and promote when ready.

Release information

Project: Log4j

Label: logging-log4j-2.25.4

Created: 2026-03-09 17:12:21

Revision: 2026-03-09 17:12:21 by pkarwasz

Upload files | Download links | Revisions | Start voting | More actions

Files in revision 00001

This draft does not have any files yet.

Distribution

While this release is in draft, you can create a staging distribution. Use the button below to record a manual distribution performed outside of ATR.

Record a manual distribution

More actions

Ignored checks

No ignored checks found.

You can also manage which check results are ignored.

Delete this draft

Permanently delete this release candidate draft and all associated files. This action cannot be undone.

Delete this draft

Apache Trusted Releases

Machine-readable artifacts

Machine-readable security metadata

We now publish:

- SBOM (CycloneDX) to Maven Central. [QA.01]
  - Useful for license compliance: all dependencies are under OSS. [LE.02]
  - Not very useful for vulnerability analysis: conflict resolution can modify transitive dependency versions.
- Vulnerability Disclosure Report: an machine-readable version of our CVE list, linked from the SBOMs. [VM.02]

Future enhancements

Common Lifecycle Enumeration (ECMA-428): **[EL.01 machine-readable]**

- Machine-readable equivalent of our list of supported versions.
- Will be generate ASF-wide through **Apache Trusted Releases**.
- We plan to introduce **Long Term Support** branches.

Vulnerability Exploitability eXchange (CycloneDX, OpenVEX, CSAF): **[VM.02+]**

- Automatically generated by VEX Generation Toolset.
- Currently tested within Apache Solr.
- Will be extended to Commons and Log4j to help downstream application with their exploitability assessment
- Guarantees “no known exploitable vulnerabilities”.

VEX Generation architecture

CVEs

SBOMs

Root Cause Service

- AI agent
- Vulnrichment
- Finds vulnerable method

Call Graph Service

- Per component call graphs
- Capslock-compatible format
- Source-based (shading-resistant)

Metadata GitHub Repository

Project-specific VEX workflows

- Determines maximum reachable path
- Generates enriched VEX statement (VEXplanation)

Maintainers

Vulnerability handling

Vulnerability handling

- CVE policy maintained by the Apache Software Foundation.
- Statistics are available in the Security Team monthly reports
- Apache Logging Services asked for a distinct reporting address: security@logging.apache.org **[VM.01]**

Since 2024:

- Explicit **Treat Model** to separate security concerns between Log4j and consumers.
- **Bug bounty program**, funded by the Sovereign Tech Resilience program.

Bug Bounty Program

- Hosted at YesWeHack: https://yeswehack.com/programs/log4j-bug-bounty-program
- Covers Log4cxx, Log4j and Log4net
- July 2024–November 2025: 32 reports, 3 CVEs disclosed (10%)
- December 2025–now: 90 reports, much lower number of valid reports, loosely classified as:
  - Many out of scope according to our security model,
  - (starting March 2026) Many duplicates,
  - Usually point to real behavior, but which does not constitute a security concern,
  - Some valid.

Limit attack surface

Log4j 2.x was conceived with pre-Maven environments in mind:

- Monolithic `log4j-core`,
- Many **optional** dependencies.

In Log4j 3.x we have:

- Modularize most of `log4j-core`,
- Only **required** dependencies in each module.

CRA-ready community

Sustainability problems

As many OSS projects, Apache Logging Services projects are chronically underfunded:

- 2 active maintainers per project
- 70-80% of work done by 2 people
- Maintenance done in free time

The real risk: **maintainer burnout**

Contribution statistics (LF Insights, 365 days)

Log4net:

- Jan Friedrich: 74%
- Davyd McColl: 5%
- Others: 21%

Log4j:

- Piotr Karwasz: 48%
- Volkan Yazıcı: 22%
- Others: 30%

What happens if we can not contribute any more?

Explored solutions to OSS sustainability

Consulting and CRA attestations:

Offer consulting and compliance attestations for widely used libraries.

Initiative:
Open Source Economy

Consulting fees support:

- Maintainers
- Project infrastructure
- Upstream dependencies

Making maintenance needs visible:

Users often do not know the **maintenance status** of their dependencies.

Initiative:
ECMA TC-54 TG4—`CONTRIBUTING.yaml`

Goal: machine-readable description of

- Contribution needs
- Maintenance status
- Support expectations

CRA-ready community

Many OSS communities were developed in different times, were:

- Maintainers could **write** to the repository (fun),
- Software was built on maintainer-owned hardware.

These habits are still very **strongly** anchored in the community, which is often reticent to switch to:

- Maintainers can only **review** contributions (not fun),
- Software is built on CI.

This presentation

Q&A

Thanks

My wife Agnieszka and my angels:
Milena, Liliana, Natalia

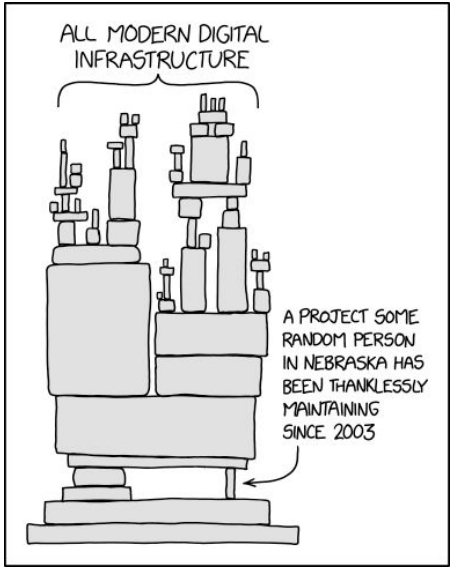
Apache Logging Services team:
C. Grobmeier, C. Kozak, D. McColl, D. Psenner,
G. Gregory, J. Friedrich, J. Katariya, M. Sicker,
R. Goers, R. Gupta, R. Popma, R. Middleton,
R. Grabowski, S. Deboy, S. Webb and Th. Schöning.
See also https://logging.apache.org

Partner in crime: Volkan Yazıcı

VEX Generation Toolset team: Munawar Hafiz and OpenRefactory

Sponsors:
AWS and Sonar

Remember about:



Source: XKCD